browser: An injected (XSS) script on www.application.com may not read the cookie for admin.application.com and vice-versa.

6 User Management

Almost every web application has to deal with authorization and authentication. Instead of rolling your own, it is advisable to use common plug-ins. But keep them up-to-date, too. A few additional precautions can make your application even more secure.

There are a number of authentication plug-ins for Rails available. Good ones, such as the populardevise and authlogic, store only encrypted passwords, not plain-text passwords. In Rails 3.1 you can use the built-in `has_secure_password` method which has similar features.
Every new user gets an activation code to activate their account when they get an e-mail with a link in it. After activating the account, the activation_code columns will be set to NULL in the database. If someone requested an URL like these, they would be logged in as the first activated user found in the database (and chances are that this is the administrator):

```
http://localhost:3006/user/activate
http://localhost:3006/user/activate?id=
```

This is possible because on some servers, this way the parameter id, as in params[:id], would be nil. However, here is the finder from the activation action:

```
User.find_by_activation_code(params[:id])
```

If the parameter was nil, the resulting SQL query will be

```
SELECT * FROM users WHERE (users.activation_code IS NULL) LIMIT 1
```

And thus it found the first user in the database, returned it and logged them in. You can find out more about it in this blog post. *It is advisable to update your plug-ins from time to time*. Moreover, you can review your application to find more flaws like this.

6.1 Brute-Forcing Accounts

Brute-force attacks on accounts are trial and error attacks on the login credentials. Fend them off with more generic error messages and possibly require to enter a CAPTCHA.

A list of user names for your web application may be misused to brute-force the corresponding passwords, because most people don't use sophisticated passwords. Most passwords are a combination of dictionary words and possibly numbers. So armed with a list of user names and a dictionary, an automatic program may find the correct password in a matter of minutes.

Because of this, most web applications will display a generic error message "user name or password not correct", if one of these are not correct. If it said "the user name you entered has not been found", an attacker could automatically compile a list of user names.

However, what most web application designers neglect, are the forgot-password pages. These pages often admit that the entered user name or e-mail address has (not) been found. This allows an attacker to compile a list of user names and brute-force the accounts.

In order to mitigate such attacks, *display a generic error message on forgot-password pages, too*. Moreover, you can *require to enter a CAPTCHA after a number of failed logins from a certain IP address*.